

방성민 (Elian)

서비스 개발부터 성능 개선과 안정적인 운영까지 담당하는 풀스택 개발자

010-7706-0986 bangseongmin@naver.com GitHub Blog TIL

INTRODUCE

병원 대상 의료 IT 서비스의 알림·CRM·통계 도메인을 담당해 온 4년차 풀스택 개발자입니다. Java·Spring Boot 백엔드가 주력이고, 운영자·병원이 쓰는 화면은 Vue 로 직접 만들었습니다. 2023년부터 알림을 맡아 메인 서버에 있던 발송 기능을 RabbitMQ 기반 별도 서비스로 옮기고, 알림톡 등 새 채널을 추가했습니다. 조회 성능과 락 경합 문제는 원인을 확인해 고쳤습니다. 발송 결과가 실제와 다르게 집계되거나 실패가 표시되지 않는 문제도 찾아, 화면과 알람에서 확인할 수 있게 했습니다. 운영에서 겪은 문제는 개인 실험으로 다시 만들어 기술 선택의 조건과 한계를 확인합니다.

Seongmin

알림 서버 분리와 채널 확장

메인 서버에 있던 발송 기능을 RabbitMQ 기반 별도 서비스로 옮기자고 제안하고 구현했습니다. 기존 발송 호출을 바꾸지 않고 알림톡을 추가했고, 새 채널도 같은 방식으로 추가할 수 있게 했습니다. 월 기준 SMS는 35,194건에서 14,821건으로 줄고, 알림톡은 4,517건이 발송됐습니다. 알림톡을 쓰는 구간의 건당 단가는 SMS보다 약 70% 낮았습니다.

RabbitMQ MSA System Design

진료 통계 KPI 서비스 0 → 1

수기로 관리하던 진료 성과 지표를 서비스로 만들었습니다. 사업팀과 지표를 정하고, 기준일을 바꿔 보는 통계도 처리하도록 구현했습니다. JVM 1Core·1GB 환경의 동시 사용자 100명 시험에서 평균 응답 시간은 26.7초에서 2.73초로, 처리량은 3.33에서 35.9 TPS로 개선됐습니다.

Data Pipeline Batch Product

락 경합의 원인을 커밋 시점까지 좁힘

주 2~3회 발생하던 lock wait timeout의 원인이 트랜잭션 길이가 아니라 완료 표시가 커밋되는 시점임을 확인했습니다. 완료 표시는 건별로 바로 커밋하게 바꿔 0건으로 줄였습니다. 이후 같은 상황을 개인 실험으로 다시 만들고 변경안별 결과를 비교했습니다.

Transaction Troubleshooting MySQL

EXPERIENCE

TOTAL 총 3년 10개월

2023. 02 ~ **(주) 플라잉닥터** 재직 중 3년 10개월

풀스택 개발자

- 병원 대상 의료 IT 서비스의 알림·CRM·통계 도메인을 백엔드 중심으로 담당하고, 운영자·병원용 화면은 프론트까지 직접 구현
- 2023년에는 통합 클라이언트와 화면을 만들었고, 2024년에는 Vue3·TypeScript 전환과 문자 과금 기능을 개발했다. 2025년부터는 통계 서비스와 알림 서버를 맡았으며, 알림 도메인은 2023년부터 계속 담당했다
- CRM 이벤트 기반 자동발송, 발송 이력 관리 및 포인트 기반 문자 과금 기능 개발
- 템플릿 데이터 구조를 설계하고 셀프서비스 편집 기능을 개발해, 이미지 제작 관련 CS 요청을 주 10건에서 2~3건으로 줄임
- 기획·사업팀과 통계 요구사항 및 지표를 정의하고, 집계 파이프라인과 조회 API 설계
- 운영 장애 분석, 데이터베이스 조회 최적화 및 처리 결과 모니터링 개선
- 주간 회의에서 설계 방향을 제안·합의하고, 배포 범위와 일정을 기획·디자인과 협의해 결정. 배포 전 변경 내용을 CS팀에 사전 공유하는 절차로 운영
- 2026년부터 담당 범위 일부를 다른 개발자에게 넘기고, 해당 프로젝트의 이관을 돕는 역할로 참여 범위를 조정

PROJECT

2025 ~ **알림 서비스 분리 및 발송 채널 확장**

(주) 플라잉닥터 · 서버 분리 제안·구현 / 채널 확장 설계·구현

- 문제 — 알림 발송이 메인 서버에 있어 대량 발송 때 서비스 전체가 느려졌고, 채널을 바꾸려면 발송 기능을 쓰는 모든 서비스를 수정해야 했다
- 규모 — SMS·알림톡·PUSH 합계 월 2만 건대(일 평균 496건). 오전 07~08시에 SMS 물량의 47.4%가 몰린다
- 발송은 메인 서버와 분리하고, 새 채널은 기존 발송 호출을 바꾸지 않고 추가할 수 있게 만들었다
- **2025 · 알림 서버 분리**
  - 팀 리뷰 뒤 RabbitMQ 기반 별도 서비스로 분리하고, 외부 API 호출은 Virtual Thread로 동시에 처리
  - 선택한 이유 — 처리량보다 메인 서버 격리와 실패 재처리가 필요했다. ACK·재처리·DLQ를 지원하는 RabbitMQ를 선택했다
  - 외부 채널사 응답을 기다리는 호출이 많아, 스레드 수를 크게 늘리지 않고 동시에 처리할 수 있는 Virtual Thread를 적용
  - 조회와 저장은 묶어서 처리하도록 바꿔 배치 작업을 줄임
- **2025.11 · 알림톡 채널 확장**
  - 기존 발송 호출을 바꾸지 않고 ChannelSender 구현체를 추가해 알림톡을 연결. 이후 새 채널도 같은 방식으로 추가할 수 있게 함
  - 알림 유형마다 알림톡을 먼저 보내고 실패하면 SMS로 넘김. 템플릿 변수 치환과 검증은 채널과 상관없이 공통으로 처리
  - 알림톡을 쓰는 전환 구간에서 건당 단가가 SMS보다 약 70% 낮음을 확인
  - 전환 실측 (2025-08 → 2026-07, 월 기준) — SMS/LMS 35,194건 → 14,821건(−58%), 알림톡 0건 → 4,517건
- **2026 · 템플릿 셀프서비스**
  - 템플릿을 코드에서 DB로 옮기고, 병원이 등록한 템플릿을 운영자가 검수하게 함
  - NCP 채널·템플릿 조회를 분리하고, OTP 인증은 알림톡을 먼저 보내고 실패하면 SMS로 보내게 함

2026 ~ **발송 신뢰성 — 결과 판정과 집계 바로잡기**

(주) 플라잉닥터 · 기획과 범위 합의 / 설계·구현

- 문제 — 발송 결과 판정과 집계가 모두 실제와 달랐다. 새 기능의 결과 판정은 세 번 고쳤고, 기존 13개월치에서도 빠진 실패를 찾았다
- **2026 · CRM 알림톡 발송 — 초기 구현을 세 번 되돌림 (STAG 반영, 운영 배포 전)**
  - 문제 — CRM 개별·단체 문자 화면에서 승인된 템플릿으로 알림톡을 보내 달라는 요청이 있었다. 새 화면을 만들지 않고 기존 발송 화면에 수단을 추가하기로 합의했다
  - 되돌림 1 — 결과 판정을 위해 발송을 동기 REST 로 바꿀 때 처음에는 모든 호출자를 한꺼번에 전환했다. OTP 와 기존 알림까지 큐의 버퍼링을 잃는 전역 변경이었고 합의된 범위도 아니어서 되돌렸다. 결과 판정이 필요한 CRM 발송만 동기로 두고 나머지는 큐를 유지
  - 되돌림 2 — 발송 결과를 성공·실패 두 값으로 판정했다가, "안 나갔다" 와 "나갔는지 모른다" 를 구분하지 못해 개발 환경에서 같은 문자가 반복 발송됐다. SENT / NOT_SENT / UNKNOWN 3분류로 바꾸고 재시도는 안 나간 것이 확실한 경우만 하도록 제한. 모르는 건은 자동 복구하지 않고 사람이 판단하게 남겼다
  - 되돌림 3 — 캠페인 전체를 한 트랜잭션으로 묶었더니 1건 실패가 이미 나간 나머지의 결과 기록까지 롤백시켰다. 예약 1건 = 트랜잭션 1개로 격리
  - 사고 건수는 처음 48회로 보였지만 실제로는 9회였다. 무효로 판정한 실험 2건도 기록에 남겼고, QA 점검표 104항목을 만들었다
  - 배포 범위 — 기다리는 병원이 있어 범위를 쪼개서라도 일정을 앞당기자는 제안이 있었으나, 전송 결과를 확인할 화면 없이 발송 기능만 먼저 내보내는 것은 부적절하다고 보고 전송 내역과 함께 내자고 했다. 일정은 지키고 범위를 묶는 쪽으로 정리
- **2026 · 집계에서 빠져 있던 실패를 드러냄**
  - 발송 결과는 DB에 있었지만 조회 화면이 없어, 물량은 매번 SQL로 확인했고 실패는 고객 문의로 알았다. 13개월치를 직접 조사했다
  - 성공 판정이 틀려 있었다 — 벤더가 거절한 141건도 상태가 COMPLETED 였다. 판정 기준을 상태값에서 응답 코드로 바꿨다. 상태 허용 목록에 빠져 있던 실패 242건도 13개월간 모든 집계에서 누락돼 있었고, 목록 밖 값을 세는 감시 쿼리를 짝으로 두는 규칙으로 정리
  - 결과 콜백을 못 받아 미확정으로 남은 건이 435건, 가장 오래된 것은 442일 경과 상태였다
  - 선택한 이유 — 발송량이 0이면 실패율도 0%로 보일 수 있다. SMS 물량의 47.4%가 오전 한 배치에 몰려 있어, 그 배치가 실패하면 실패율만으로는 문제를 찾을 수 없다

2025. 02 ~ 2025. 07 **통계 집계 및 KPI 리포트 서비스 개발**

(주) 플라잉닥터 · 기획 제안 / 사업팀과 지표 정의 / 집계 설계·구현

- 문제 — 병원은 직원·의사 성과를 수기로 관리했고, 재방문·이탈·피크타임은 감각으로 판단했다
- **수기로 관리하던 진료 성과 지표를 집계 서비스로 대체**
- 병원별·진료과목별 차이를 설정 값으로 모델링해 공통 집계 로직을 재사용
- EMR을 직접 연결할 수 없어, 병원이 내려받은 Excel 데이터를 올리면 직원·의사별 지표를 계산하게 함
- 통계 조회는 운영 서비스와 다른 데이터소스로 나누고, 대용량 데이터는 한꺼번에 메모리에 올리지 않고 처리
- 선택한 이유 — 환자 수와 재방문 수는 사용자가 기준일을 바꿔 보므로 미리 집계할 수 없었다. 원본 데이터를 반영하는 집계 테이블과 조회용 인덱스를 따로 설계했다
- **JVM 1Core·1GB 환경의 동시 사용자 100명 시험에서 평균 응답 26.7초 → 2.73초, 처리량 3.33 → 35.9 TPS. 단일 호출 평균 응답은 0.76초 → 0.163초**
- 별도 50명·10초 타임아웃 시험에서는 일별 통계 집계 실패율이 86% → 0%가 됐고, 통계 갱신 주기는 1일 1회 → 1시간으로 단축

2026 ~ **배치 외부 호출의 worker 이관과 효과 검증**

(주) 플라잉닥터 · 원인 분석·설계·구현 주도

- 외부 이미지 추출을 worker로 옮기고, 속도 차이의 원인을 비교 실험으로 확인했다. 이후 실패로 표시되지 않던 썸네일 누락도 찾았다
- **① 배치의 외부 호출을 worker 로 분리**
  - 문제 — 일배치가 병원 도메인에 HTTP 요청 약 440건을 순차로 보내 대표 이미지를 가져오면서 오래 걸렸다. APM에서 이 구간이 22초 이상임을 확인했다
  - 배치는 요청만 등록하고, 별도 worker가 이미지를 가져와 결과를 반영하게 했다. 다시 실행해도 같은 요청을 중복 처리하지 않게 하고 외부 URL 접근도 제한했다
  - stag APM에서 이미지 추출은 92.5초 → 13.3초였다. 같은 기간 이 변경이 없는 운영 환경은 149초대를 유지했다
- **② 빨라진 원인을 벤치마크로 규명**
  - Java와 Node의 동기·비동기 방식 4가지를 페이지 수와 동시성 6단계에서 각 5회 비교했다. 추출 로직과 mock 서버는 고정했다
  - Java 동기와 Node 동기는 0.6% 차이였다. 언어보다 순차 처리와 비동기 처리의 차이가 컸다
  - mock 서버 기준으로 Node 비동기는 같은 처리량에서 CPU를 약 37.7% 적게 썼다. worker를 나눈 이유는 언어가 아니라 동시성, 격리, CPU 효율로 정리했다
- **③ 실패로 잡히지 않는 누락을 관측 지표로**
  - 실패·폴백 카운트는 모두 0인데, 썸네일 적용률은 86~93%에서 14~27%로 떨어졌다. 11일치 요청 기록에서 worker가 빈 값을 자주 돌려준 것을 찾았다
  - 적용률 하락은 worker 인스턴스 교체 시점과 겹쳤지만, 기술적 원인은 확인하지 못했다. 로그 3일치만 보면 평균 76%로 보였지만, 11일 기록으로는 평균 54%였다
  - 전체 요청 중 썸네일이 적용된 비율과 추출 실패율을 알람으로 삼았다. 대상 페이지 5,057건 중 4,825건에 썸네일이 적용된 상태(95.4%)

2023 ~ 2026 **CRM 자동발송과 포인트 기반 문자 과금**

(주) 플라잉닥터 · 과금 설계·구현 / 자동화 설계·구현 / 알림 전수 조사

- 문제 — CRM 발송은 사람이 대상을 골라 매번 직접 실행했고, 문자 발송에는 과금 수단이 없었다. 2023년 CRM 화면, 2024년 과금, 2026년 자동화으로 이어진 작업이다
- **포인트 차감 방식의 문자 과금을 구현. 충전·환불과 중복 발송 방지를 넣어 과금 없이 발송되는 경로를 막음**
- **특정 이벤트 뒤 N일이 지나면 자동 발송하는 조건을 만들고, 1회성 발송 미리보기는 별도로 처리**
- 진료기록 키워드, 이탈·재방문 제외 조건으로 발송 대상을 고르고 추천 대상도 관리
- PUSH·알림톡·SMS 발송 목록을 조사해 앱 푸시 기준 20개 알림을 정리 (절감액은 측정하지 않음)

2024 ~ 2025 **홈페이지 이미지 편집 셀프서비스**

(주) 플라잉닥터 · 기획·CS팀과 요구 정의 / 템플릿 구조 설계 / 편집기 구현

- 문제 — 병원이 홈페이지 이미지를 바꾸려면 CS팀이 직접 만든 뒤 전달해야 했고, 요청마다 며칠을 기다렸다
- 텍스트와 이미지를 나눈 템플릿 데이터와 저장·조회 API, 템플릿 버전 관리를 구현
- Fabric.js 기반 편집기를 구현. 편집 상태와 화면 표시를 나누고, 바뀐 객체만 다시 그리게 함
- **도입 후 이미지 제작 관련 CS 요청 주 10건 → 2~3건, 같은 기간 병원이 직접 하는 초기 셋업은 주 2~3건 → 5건**

2024 ~ 2026

조회 성능 및 락 경합 개선

(주) 플라잉닥터 · 원인 분석·설계·구현

- **2024.11 · 리마인드 알림 조회 최적화**
  - EXPLAIN ANALYZE로 날짜 계산 때문에 인덱스를 못 쓰는 것을 확인하고, 범위 조건으로 바꿔 인덱스를 다시 사용하게 함
  - 선택한 이유 — 새 인덱스를 추가하지 않고 기존 복합 인덱스를 활용해 쓰기 성능을 유지
  - 리마인드 알림 조회 응답 시간을 약 5~6초에서 300~400ms로 단축
- **2025 · 조회 방식 셋을 부하테스트로 비교해 선택**
  - 문제 — 통계의 인기 페이지 목록이 링크만 보여 어떤 페이지인지 알기 어려웠다. 대표 이미지를 보여 주기 위한 조회 방식을 정해야 했다
  - 요청 때 외부 페이지를 읽는 방식, 테이블에 저장해 두는 방식, 저장 후 Redis로 캐싱하는 방식을 같은 조건에서 비교. Redis 자료구조도 String과 Hash로 나눠 비교
  - 외부 페이지를 직접 읽는 방식은 170건이 모두 타임아웃이었다. 저장 방식으로 바꾸자 실패가 사라졌고, Redis를 더하자 평균 633ms → 399ms가 됐다. 동시 사용자 10명 조건에서 Hash 이점은 없어 String을 선택
- **2026 · 푸시 발송 락 경합 대응**
  - 발송 배치가 콘텐츠 행을 잡은 채 커밋하지 않아 상세 조회가 lock wait timeout으로 실패했다. 완료 표시가 커밋되는 시점을 원인으로 확인해 해당 갱신을 건별 커밋으로 분리
  - 반복되던 락 경합 timeout 주 2~3회 → 0건 (발송 대상 평균 1천 명 규모)

SIDE PROJECT

알림 발송 환경의 트랜잭션·락 경합 재현

- 운영에서 겪은 락 경합을 다시 만들고, 트랜잭션 경계와 갱신 방식을 바꿔 가며 결과를 비교한 개인 실험
- 완료 표시를 건별로 커밋했을 때만 실패가 0건이었다. 다른 변경안은 효과를 확인하지 못했고, 판단할 수 없는 범위도 저장소에 적었다
- notification-reliability-lab · GitHub (https://github.com/Elian-bang/notification-reliability-lab)

리뷰 규칙을 CI 가 막게 만들기

- 장애에서 나온 규칙 6개를 ArchUnit 으로 구현하고, 같은 PR에서 위반 커밋은 차단되고 위반을 제거한 커밋은 통과하는 것을 확인한 개인 실험
- 규칙도 테스트로 검증했다. 첫 실행에서 클래스만 검사하는 규칙과 메서드에 붙인 위반 예시가 맞지 않는 것을 찾았다
- 간접 호출은 검사 범위 밖이라고 표시했다. 규칙마다 위반 예시가 하나뿐이라 실제 코드의 탐지율과 오탐률은 아직 측정하지 않았다
- backend-guardrails · GitHub (https://github.com/Elian-bang/backend-guardrails)

Virtual Thread 적용 조건 측정

- 플랫폼 스레드와 가상 스레드를 조건 26개에서 각 3회 비교하고, 결과를 확인하기 위해 JDK와 JDBC 드라이버 조건도 바꿔 235회를 측정
- DB를 거치지 않는 경로에서는 유리했지만, DB를 거치는 경로에서는 결과가 뒤집혔다. 캐리어 수에 민감한 것은 확인했지만 드라이버가 원인인지는 아직 확인하지 못했다
- virtual-thread-lab · GitHub (https://github.com/Elian-bang/virtual-thread-lab)

SKILL

Language Java TypeScript

Backend	Spring Boot · Spring Data JPA · QueryDSL · MyBatis · Spring Batch
Database & Messaging	MySQL · Redis · RabbitMQ
Infrastructure & Ops	Docker · Kubernetes · GitHub Actions · Spring Boot Actuator · Elastic APM · Git
Frontend	Vue.js 2·3 · TypeScript · Fabric.js

EDUCATION

2016. 03 ~ 2022. 08 — **우송대학교**
IT융합학부 컴퓨터정보보안전공 학사 졸업 (학점 4.17/4.5)

2021. 08 ~ 2022. 02 — **비트교육과정 (고급)**
Windows .NET(C#·WPF·ASP.NET), 모바일 결제·인증 통합 앱 및 WebRTC 화상회의 웹 개발

2021. 03 ~ 2021. 05 — **비트교육과정 (단기)**
C · C++ · Java · 자료구조

ETC

2021. 12 — **우수상**
우송대학교 창업지원단

2021. 12 — **전국 SW 알고리즘 대회 독행상**
우송대학교 · 구름IDE

2021. 05 — **단정상**
우송대학교 SW사업단

2017. 11 — **장려상**
NIA 한국인터넷진흥원 · SK텔레콤